

Assignment 7 (Week 7): Doubly & Circular Lists, List-Backed Stack and Queue, and Polynomials

Auqib Hamid Lone

Department of Computer Science & Engineering, GCET Ganderbal

October 3, 2026

Due: two weeks from issue. There is no automatic late penalty — if you need more time, ask before the deadline and an extension will be granted (see the course policy in the syllabus).

Answer every question. Show all working for Numerical and Design questions. Each problem states its own assumptions; everything you need is in Week 7’s Notes chapter alone. Notation matches the lecture exactly: `struct node { int data; struct node *prev; struct node *next; };`, `head` is the first-node pointer (NULL when empty), `p->prev/p->next` are the predecessor/successor links, NULL is uppercase, a circular list satisfies `last->next == head` with `head->prev` the last node, list-based stack has `top = head`, list-based queue removes at `head` and appends at `tail`, polynomial nodes are `struct pnode { float coeff; int exp; struct pnode *next; };` kept in strictly descending exponent order, and positions are 0-based. For every bound, name the operation, give the bound, and state the case (Week 2 shape).

7.1 Conceptual

Q1. A photo editor keeps its undo states in a linked structure. “Undo” deletes the most recent state; “revert-to” jumps to an older state and discards every state after it; in both cases the editor already holds a pointer straight to the node it must remove (Karumanchi Ch. 3, pp.74–162 PDF; Wirth Sec. 4.3, p. 115) [3, 5].

- (a) State the **two-link invariant** for an interior node `p` of a doubly linked list (DLL), then name the two links that a delete-by-pointer must repair and write the two bypass assignments (with their guards).
- (b) Explain why the same delete is $O(n)$ worst case in a *singly* linked list but $O(1)$ worst case in a DLL once `p` is known — name precisely the information the backward link supplies that a singly list must re-find by walking.
- (c) Give the honest bill in one or two sentences: what the second link costs in space and in per-operation maintenance, and why search is unchanged (give both of its bounds and the case). Close with the workload rule for choosing a DLL over a singly list.

Why this matters: the backward link’s payoff is the week’s whole point, and it is only real if you can state the invariant and price the second link together — this question makes you do both on the editor’s own workload.

Q2. A round-robin scheduler stores its runnable processes in a circular list and hands out the CPU by walking the ring (Karumanchi Ch. 3, pp.74–162 PDF) [3].

- (a) A singly circular list reads `head → 9 → 18 → 27 → head`. Give `9->next`, `27->next`, and the sentinel that replaces `NULL`. If the same ring is made doubly circular, what is `head->prev`, and what does that buy?
- (b) The deck's trap: in one or two sentences, explain why `while (p != NULL) { p = p->next; }` never terminates on this ring. Then give the correct terminating traversal that prints 9 18 27 exactly once, and say in one sentence why the `do--while` form (not a plain `while (p != head)`) is required.
- (c) For an n -node singly circular ring, how many link steps does one full traversal take, and what is its bound (name the operation, the bound, and the case)? In one sentence, why must every insert/delete on a circular list repair a closure link that a `NULL`-terminated list does not have?

Why this matters: a scheduler that waits for `NULL` spins forever — this question pins the termination rule to the exact sentinel and makes you pay the closure repair before you write any ring code.

7.2 Numerical

Q3. Start from the DLL `head ↔ 8 ↔ 16 ↔ 32 ↔ 64`. Rules: insert after a given node uses the deck's four-write order (`newNode->next`, `newNode->prev`, `old->prev`, `p->next`); delete-by-pointer uses the guarded bypass then `free` exactly once.

- (a) Insert 24 after the node holding 16. Give the four assignments in order, the final chain, and verify both halves of the two-link invariant at the 24-node.
- (b) On the result, delete the node holding 32, given only a pointer `p` to it. Write the two bypass assignments (with their guards), name the freed node, give the final chain, and verify the invariant at the 24-node (now adjacent to 64).
- (c) On the result of (b), delete the head node (8), given only a pointer to it. Which guard changes the code path, what is the bypass, and what is the final chain? State the complexity for (a), (b), and (c) with operation, bound, and case.

Why this matters: insert-after, delete-middle, and delete-head are the same four-write surgery in three landings — this question puts your hand through all three on one chain the lecture never traced, and asks you to certify each with the invariant.

Q4. Two interfaces on one implementation: the Week 3 stack and the Week 5 queue contracts, now on chains. Stack top is `head`; queue removes at `head` and appends at `tail`.

- (a) List-based stack: start `head → 7 → 4 → 1 → NULL` (top 7). Run `push(9)`, `pop`, `pop`, `push(5)`, `pop`. Give the chain after every step as a five-row table, count the total link writes (`next` assignments plus `head` moves), and state the complexity of `push/pop` with operation, bound, and case.
- (b) List-based queue: start `head → 2 → 4 → 6 → NULL` with `tail` at 6. Run `dequeue`, `dequeue`, `enqueue(8)`, `dequeue`, `dequeue`. Give `head`, the chain, and `tail` after every step, and name the exact step where the `if (head == NULL) tail = NULL` guard fires. State the complexity of `enqueue/dequeue` with operation, bound, and case.
- (c) A singly circular ring holds n processes. How many link steps does one full traversal take, and what is the bound (operation, bound, case)? In one sentence, why does the linked stack/queue avoid the fixed-capacity or amortized-reallocation cost the array version pays?

Why this matters: the contract does not change when the implementation does — this question traces both interfaces on chains and makes you price the one edge case (the emptying queue) where the linked version needs a guard the array never did.

7.3 Design

Q5. Design polynomial addition over ordered (`coeff`, `exp`) chains (the lecture’s `struct pnode`), then extend it to multiplication. Convention: strictly descending exponent order; zero-coefficient terms are never stored (Horowitz & Sahni Ch. 4, standard treatment; Aho–Hopcroft–Ullman Ch. 2, standard treatment) [2, 1].

- (a) Write pseudocode `polyAdd(A, B)` returning a new chain `C` in descending exponent order. Walk both lists once; a strictly larger exponent is copied through; equal exponents add coefficients and emit a node only when the sum is nonzero; append the remainder of whichever list is not exhausted. Keep a `C`-tail pointer so each append is $O(1)$, and include the empty-chain and `malloc`-failure checks.
- (b) Trace `polyAdd` with $A = 6x^5 + 3x^2 + 4$ and $B = -6x^5 + 2x^3 + 3x^2 + 9$: give a step table (compare $\rightarrow$ action $\rightarrow$ `C` so far), name every term that is cancelled and therefore not stored, and give the final chain.
- (c) State the worst-case time bound of `polyAdd` with its independent variables, and why. Then state the worst-case bound of the multiplication routine (each `A`-term times every `B`-term, exponents added, equal exponents merged) with its independent variables, and in one sentence why multiplication is not linear while addition is.

Why this matters: sparse polynomials are the syllabus’s showcase for “store only what exists” — this question designs the linear merge and then prices the quadratic extension, so you can defend the representation’s cost, not just its shape.

Q6. Design a *list-based circular queue* that keeps the ring closed with `tail->next == head`, so a single `tail` handle reaches both ends (front = `tail->next`, rear = `tail`) (Karumanchi Ch. 5, pp.205–223 PDF; Sedgewick Sec. 1.3, p. 120) [3, 4].

- (a) Write pseudocode for `enqueue(x)` and `dequeue()` on this ring. Handle all three cases explicitly: empty ring, single-node ring (`dequeue`), and the general ring. No `count`, no sacrificed slot, no `mod` — say in one sentence how the single `tail` pointer distinguishes full from empty where a ring over array indices could not.
- (b) Starting empty, run `enqueue(11)`, `enqueue(22)`, `enqueue(33)`, `dequeue`, `dequeue`, `enqueue(44)`, `dequeue`, `dequeue`. Give the ring contents (front to rear) and the location of `tail` after every step.
- (c) State the complexity of `enqueue` and `dequeue` with operation, bound, and case, and compare in one sentence with the Week 5 array circular queue’s full/empty test.

Why this matters: Week 5 spent a `count` field or a sacrificed slot to tell full from empty; a closed ring spends neither — this question designs the trick and traces the one case (the last `dequeue`) where it must collapse the ring.

References

- [1] Alfred V. Aho, John E. Hopcroft, and Jeffrey D. Ullman. *Data Structures and Algorithms*. Addison-Wesley, 1st edition, 1983.
- [2] Ellis Horowitz and Sartaj Sahni. *Fundamentals of Data Structures*. Galgotia Booksources, 2nd edition, 2008.
- [3] Narasimha Karumanchi. *Data Structures And Algorithms Made Easy*. CareerMonk Publications, 2017. This calibre-generated edition carries no printed folio numbers; page numbers cited in CS301 material are PDF page indices. NOT a source for Week 1’s C-specific pointer/malloc content — see this course’s supporting-books index for the coverage check.
- [4] Robert Sedgewick and Kevin Wayne. *Algorithms*. Addison-Wesley, 4th edition, 2011.
- [5] Niklaus Wirth. *Algorithms and Data Structures*. Author (freely distributed), 2004. Oberon version, August 2004; revision of the 1985 Prentice-Hall edition. Page numbers cited in CS301 material refer to this Oberon PDF, not to the 1985 print edition.